1 Lecture 16 (Oct 20) - Graphs: Definitions, Representations, Search, Reductions

For the next two or two and a half weeks: fundamental **graph algorithms**. You will rarely design a new graph algorithm on homework. Instead you will **reduce** problems to these algorithms: build a graph, run a known algorithm on it, and read off the answer. Lecture explains how the algorithms work; homework uses them.

1.1 What Is a Graph?

A drawing of dots and curves is only a picture. Two very different drawings can show the same graph. The definition is more abstract:

A **graph** is an ordered pair $G = (V, E)$ of two sets.

- V is any non-empty, finite set: the **vertices** (or nodes). Same idea as an alphabet.
- E is the set of **edges** (or arcs). What an edge is depends on the kind of graph:

Kind	**An edge is**	**Written**
Directed	an ordered pair (u, v) with $u, v \in V$	$u \to v$
Undirected	an unordered pair $\{u, v\}$: a set of exactly two vertices	uv

The short notation keeps descriptions of edge sets readable without nested brackets.

1.1.1 Vertices can be anything

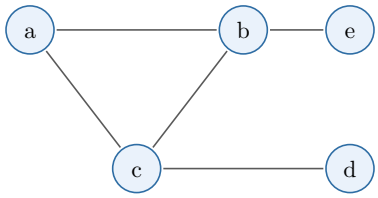
V is any finite set, so you can make the **objects of a problem** into vertices. This is the key to reductions.

Graph	**Vertices**	**Edges**
Intersection graph	Geometric objects (segments, disks)	Pairs of objects that intersect
Dependency graph	Distinct subproblems of a recursive algorithm	$u \to v$ if u calls v directly. Fibonacci: $F_6 \to F_5$ and $F_6 \to F_4$
Configuration graph	States of a game, puzzle or machine (for example, every legal placement of 4 Tower of Hanoi disks)	Pairs one move apart. Directed if a move can't be undone
NFA state graph	States	Possible transitions on one symbol

"Can this puzzle get from configuration A to configuration B?" is exactly **"is there a path from A to B in the configuration graph?"** The path is the sequence of moves. Dependency graphs come back soon, as another way to see dynamic programming.

1.2 Representing a Graph

Assume G is **simple**: no parallel edges (never two edges from u to v, or two edges with endpoints u and v). Example, undirected:



	a	b	c	d	e
a	0	1	1	0	0
b	1	0	1	0	1
c	1	1	0	1	0
d	0	0	1	0	0
e	0	1	0	0	0

a	b → c
b	a → c → e
c	a → b → d
d	c
e	b

The graph, its adjacency matrix, and its adjacency list.

1.2.1 Adjacency matrix

A $|V| \times |V|$ array, indexed by vertices. Entry (u, v) is 1 if $u \to v \in E$ (directed) or $uv \in E$ (undirected), and 0 otherwise. An undirected edge gives **two** 1s, at (u, v) and (v, u).

1.2.2 Adjacency list

A 1D array indexed by vertices. Entry u is a list of its **neighbors**: every v with $u \to v \in E$ (directed) or $uv \in E$ (undirected). A linked list is fine; nothing fancier is ever needed.

Operation	**Adjacency matrix**	**Adjacency list**
Is $u \to v$ an edge?	$O(1)$	$O(\deg(u))$: walk u's list
List the neighbors of u	$O(\|V\|)$: walk the whole row	$O(1)$ per neighbor
Space	$O(\|V\|^2)$, even with no edges	$O(\|V\| + \|E\|)$

From now on, assume adjacency lists and give running times in terms of $|V|$ and $|E|$. Graph algorithms almost always ask "what are u's neighbors?" and almost never ask "is uv an edge?", so the slow edge test doesn't matter.

An intersection graph behaves like an **implicit** adjacency matrix: you can test whether two objects intersect in $O(1)$, but you can't list an object's neighbors without checking every other object.

1.3 Reachability and Whatever-First Search

Reachability: Given a graph $G = (V, E)$ (directed or undirected) and a vertex s, find every vertex v that s can **reach**: there is a path from s to v.

- **Directed:** the path must start at s and end at v. v might not be able to get back to s.

- **Undirected:** paths go both ways, so this finds the **connected component** of s: the largest set of vertices with a path between every pair.

The adjacency list only tells you about **single edges**. A path can use many edges, so finding them takes work.

1.3.1 The idea

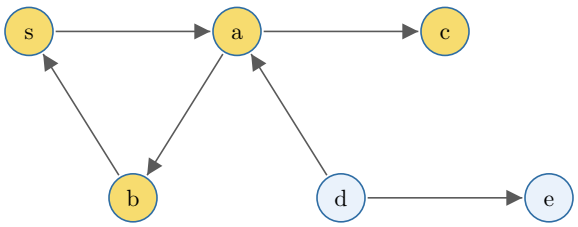
Depth-first search (DFS) and breadth-first search (BFS) both solve reachability. They do the **same important things**, and differ only in an implementation detail. So describe one general algorithm, **whatever-first search**:

1. s can reach itself (a path with no edges).
2. If s can reach u and $u \to v$ is an edge, then s can reach v. So every time you find a reachable vertex, remember to search out of it later.
3. A cycle could send the search around forever. So **mark** a vertex when you explore its edges, and never explore it again.

A **bag** is any data structure that supports **insert a vertex** and **remove some vertex** that was inserted. Which vertex comes out depends on the bag.

```
WhateverFirstSearch(s):
  put s into the bag
  while the bag is not empty
    take u from the bag
    if u is unmarked
      mark u
      for each edge u → v
        put v into the bag
```

Example: A directed graph. Searching from s marks the yellow vertices. Nothing leads from them to d or e ($d \to a$ points the wrong way):



Claim: Whatever-first search from s marks **exactly** the vertices s can reach. The full proof is in Erickson's book. You won't be asked for proofs like this. Both halves use induction:

- **Marked ⇒ reachable:** Induction on the order of marking. Either $v = s$, or v went into the bag through an edge $u \to v$ from a vertex u marked earlier. By induction u is reachable, so v is too: add one edge.

- **Reachable ⇒ marked:** Induction on the distance from s to v. If $v = s$, it goes into the bag first and comes out marked. Otherwise a shortest path ends with an edge $u \to v$, where u is closer to s. By induction u gets marked. Then v goes into the bag, and it gets marked later.

1.3.2 Running time

Let T be the time for one bag insertion or removal. Count each line:

Step	Times	Why
`mark u`	$\leq \lvert V \rvert$	Only unmarked vertices get marked, and marks never go away
`for each edge u → v`	$\leq \lvert E \rvert$ total	Only right after marking u, which happens once
`put v into the bag`	$\leq \lvert E \rvert + 1$	Once for s, then once per edge seen
`take u from the bag`	$\leq \lvert E \rvert + 1$	Can't remove more than was inserted

$$\text{Total: } O(|V| + |E| \cdot T)$$

For reachability, use the simplest bag: a **stack** or a **queue**, with $T = O(1)$:

$$\boxed{O(|V| + |E|)}$$

That is as fast as possible: the time it takes just to read the graph.

Bag	Search
Stack	Depth-first search (roughly)
Queue	Breadth-first search: also finds shortest paths, counted in edges
Priority queue	Other algorithms, with a slower T (later)

Different bags find the vertices in different orders and build different **spanning trees**. More on Thursday.

1.4 Reductions

Reducing problem A to problem B: solve an instance of A by building one or more instances of B and running an algorithm for B. From now on: build a graph, then run a known graph algorithm.

1.4.1 Example: useless DFA states

Input: a DFA $M = (Q, \{0, 1\}, \delta, s, A)$ with n states. A state q is **useless** if no input string reaches it: there is no x with $\delta^*(s, x) = q$. **Goal:** find every useless state, so you can delete them. (The subset construction for NFA → DFA often makes many of them.)

Moving from state to state along transitions is like walking from vertex to vertex. So:

1. Build a graph: $G = (V, E)$ with

$$V = Q, \qquad E = \{p \to q \mid \delta(p, 0) = q \text{ or } \delta(p, 1) = q\}$$

2. Say what graph problem to solve, and why it helps:

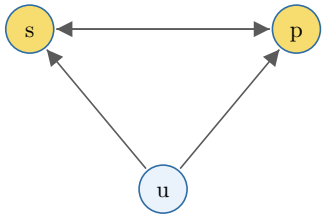
$$q \text{ is useless} \iff s \text{ cannot reach } q \text{ in } G$$

3. Say which algorithm to run, and how to use its output: Run whatever-first search from s. The useless states are the vertices it **doesn't** mark.

4. Running time, in terms of the original input: G has $|V| = n$ vertices and $|E| \leq 2n$ edges (two transitions per state). Whatever-first search takes $O(|V| + |E|)$, so the algorithm takes

$$\boxed{O(n)}$$

q	$\delta(q, 0)$	$\delta(q, 1)$
s	p	s
p	s	p
u	s	p



A small DFA and its graph (self-loops not drawn). Search from s marks s and p, so u is useless.

Key idea **That is the whole answer:** For a reduction, write down: the graph (its vertices and edges, and what they mean if it isn't obvious), the graph problem and why its answer helps, the algorithm and how to use its output, and the running time. This is exactly the detail a full-credit exam solution needs. No induction proof is needed at this point in the course.

Watch out **Say "whatever-first search", not "DFS" or "BFS":** Unless you say otherwise, it means the stack or queue version. Naming a specific search is more detail than needed, and suggests you aren't thinking about the actual problem (reachability, not shortest paths). It can lose a few points.

Watch out **Report the running time in the input's own terms:** $|V|$ and $|E|$ belong to a graph you made up. The person asking about useless states has never heard of it, so give $O(n)$, not $O(|V| + |E|)$. Building the graph is assumed to take $O(|V| + |E|)$ time. If building it is complicated, count that too.

Next time: more about the bag and spanning trees, and a reduction that builds a **new** graph out of a given graph (it's in the lecture notes on the website already, useful for the lab).